Table des matières

Cours 06 — Les codes 1
Étape 1 — Les vues des deux journaux · STAGING 1
Étape 2 — Le fait d'accumulation, `fct_lots` · MARTS 3
Étape 3 — Le fait sans mesure, `fct_consentements` · MARTS 3
Étape 4 — Lire les deux faits · MARTS 4
Étape 5 — Les tests des deux faits · MARTS 5

Cours 06 — Les codes

Cours 6/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
		✓ ici

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Chaque fichier est rangé sous son chemin, et chaque requête du notebook sous son étape ; les requêtes se tapent dans le notebook de DuckDB (`duckdb -ui`, lancé depuis `Projet_Brulerie`), une par cellule, sous celle qui attache l'entrepôt. Un fichier qui existait déjà se montre en entier, dans son nouvel état.

Étape 1 — Les vues des deux journaux · STAGING

02_ae/models/staging/_sources.yml

```yaml
# 02_ae/models/staging/_sources.yml : la RAW reçue, déclarée une seule fois
sources:
  - name: brulerie
    description: "La RAW du café, tout en texte : chaque export gardé, même deux fois."
    # 1. les tables de la RAW vivent dans le schéma par défaut de DuckDB
    schema: main
    # 2. une entrée par table qu'un modèle lit
    tables:
      - name: raw_tickets
        description: "Les tickets de la caisse, un par ligne et par export."
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
      - name: raw_lignes
        description: "Les lignes des tickets, un produit vendu par ligne et par export."
      - name: raw_produits
        description: "Le catalogue de la caisse au 2025-06-30."
      - name: raw_membres
        description: "Les membres du programme de fidélité au 2025-06-30."
      - name: raw_modifications
        description: "Le journal des modifications du catalogue, une ligne par valeur changée, depuis 2022."
      - name: raw_lots
        description: "Le journal des lots du torréfacteur, un lot par ligne, en juin 2025."
      - name: raw_consentements
        description: "Le journal des consentements des membres, une ligne par décision, depuis 2023."
```

02_ae/models/staging/stg_lots.sql

```sql
-- 02_ae/models/staging/stg_lots.sql : les lots du torréfacteur, typés, un par lot
select
    -- 1. typer, dans l'ordre du dictionnaire
    lot_id,
    debut::timestamp as debut,
    produit_id::integer as produit_id,
    profil,
    poids_vert_kg::decimal(10, 2) as poids_vert_kg,
    poids_torrefie_kg::decimal(10, 2) as poids_torrefie_kg,
    premier_craquement_s::integer as premier_craquement_s,
    duree_s::integer as duree_s,
    temp_sortie::decimal(10, 1) as temp_sortie,
    statut,
    remarque
-- 2. lire la RAW par sa source ; le journal n'a qu'un export, rien à dédoublonner
from {{ source('brulerie', 'raw_lots') }}
```

02_ae/models/staging/stg_consentements.sql

```sql
-- 02_ae/models/staging/stg_consentements.sql : le journal des consentements, typé, une ligne par décision
select
    -- 1. typer, dans l'ordre du dictionnaire
    membre_id,
    horodatage::timestamp as horodatage,
    consentement,
    statut,
    canal
```


© 2026 iAlexMG. Tous droits réservés.

```
-- 2. lire la RAW par sa source ; le journal n'a qu'un export, rien à dédoublonner
from {{ source('brulerie', 'raw_consentements') }}
```

Étape 2 — Le fait d'accumulation, `fct_lots` · MARTS

02_ae/models/marts/fct_lots.sql

```
-- 02_ae/models/marts/fct_lots.sql : le fait d'accumulation des lots, une ligne par lot, un jalon par colonne
select
    -- 1. une clé vers chaque dimension : le jour de la charge, le café torréfié
    debut::date as cle_date,
    md5(produit_id::varchar) as cle_produit,
    -- 2. le lot, gardé dans le fait, avec son profil et son statut
    lot_id,
    profil,
    statut,
    -- 3. les jalons : la charge, le premier craquement, la sortie
    debut as charge_le,
    debut + premier_craquement_s * interval 1 second as craquement_le,
    debut + duree_s * interval 1 second as sortie_le,
    -- 4. les durées entre les jalons, en secondes, puis les deux poids
    premier_craquement_s as secondes_avant_craquement,
    duree_s - premier_craquement_s as secondes_de_developpement,
    poids_vert_kg,
    poids_torrefie_kg
from {{ ref('stg_lots') }}
```

Étape 3 — Le fait sans mesure, `fct_consentements` · MARTS

02_ae/models/marts/dim_date.sql

```
-- 02_ae/models/marts/dim_date.sql : les jours du 1er mai 2023 au 30 juin 2025, un par ligne
select
    -- 1. la clé : la date elle-même
    jour::date as date_jour,
    -- 2. le nom du jour, du lundi (1) au dimanche (7)
    case isodow(jour)
        when 1 then 'lundi' when 2 then 'mardi' when 3 then 'mercredi'
        when 4 then 'jeudi' when 5 then 'vendredi' when 6 then 'samedi'
        when 7 then 'dimanche'
    end as nom_jour,
```


© 2026 iAlexMG. Tous droits réservés.

```
    -- 3. la fin de semaine : le samedi et le dimanche
    isodow(jour) >= 6 as est_fin_de_semaine
-- 4. une ligne par jour, de l'ouverture du programme de fidélité au 30 juin, les deux bornes comprises
from generate_series(date '2023-05-01', date '2025-06-30', interval 1 day) as jours(jour)
```

02_ae/models/marts/fct_consentements.sql

```
-- 02_ae/models/marts/fct_consentements.sql : le fait sans mesure des consentements, une ligne par décision
select
    -- 1. une clé vers chaque dimension : le jour de la décision, le membre qui l'a prise
    horodatage::date as cle_date,
    md5(membre_id) as cle_membre,
    -- 2. la décision, gardée dans le fait : son instant, ce qu'elle vise, ce qu'elle dit, son canal
    horodatage,
    consentement,
    statut,
    canal
-- 3. aucune mesure : le compte des lignes est l'information
from {{ ref('stg_consentements') }}
```

Étape 4 — Lire les deux faits · MARTS

4.3 Le même lot à 7 h 12

```
-- 1. le lot L-2025-0214, tel que le fait l'aurait porté à 7 h 12 : un jalon pas encore franchi reste vide
select lot_id, charge_le,
       case when craquement_le <= timestamp '2025-06-03 07:12:00' then craquement_le end as craquement_le,
       case when sortie_le <= timestamp '2025-06-03 07:12:00' then sortie_le end as sortie_le
from brulerie.fct_lots
where lot_id = 'L-2025-0214';
```

4.4 Les lots par profil

```
-- 1. par profil : les lots, la phase de développement en moyenne, la perte de poids en pour cent
select profil, count(*) as lots,
       round(avg(secondes_de_developpement)) as developpement,
       round(100 * (1 - sum(poids_torrefie_kg) / sum(poids_vert_kg)), 1) as perte
from brulerie.fct_lots
group by profil
order by developpement;
```


© 2026 iAlexMG. Tous droits réservés.

4.5 Les décisions par statut et par canal

```sql
-- 1. les décisions, par statut et par canal : le compte des lignes est la mesure
select statut, canal, count(*) as decisions
from brulerie.fct_consentements
group by statut, canal
order by statut;
```

4.6 La dernière décision de chaque membre

```sql
with derniere as (
    -- 1. de chaque membre, sa dernière décision
    select cle_membre, statut
    from brulerie.fct_consentements
    qualify row_number() over (partition by cle_membre order by horodatage desc) = 1
)
-- 2. les membres, par leur dernière décision
select statut, count(*) as membres
from derniere
group by statut
order by statut;
```

Étape 5 — Les tests des deux faits · MARTS

02_ae/models/marts/_marts.yml

```yaml
# 02_ae/models/marts/_marts.yml : les dimensions et les trois faits, décrits et testés
models:
  # 1. les dimensions : une clé unique et toujours remplie
  - name: dim_date
    description: "Les jours du 1er mai 2023 au 30 juin 2025, un par ligne, fabriqués sans source."
    columns:
      - name: date_jour
        description: "La clé : la date elle-même."
        data_tests:
          - unique
          - not_null
  - name: dim_produit
    description: "Un produit du catalogue par ligne, au 2025-06-30."
    columns:
      - name: cle_produit
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
      description: "La clé de substitution, md5 du numéro du produit."
      data_tests:
        - unique
        - not_null
    # 2. une liste fermée, et son not_null à côté
    - name: categorie
      data_tests:
        - not_null
        - accepted_values:
            arguments:
              values: ['ACCESSOIRE', 'BOISSON_CHAUDE', 'BOISSON_FROIDE', 'GRAINS', 'SANDWICH', 'VIENNOISERIE']
- name: dim_membre
  description: "Un membre du programme de fidélité par ligne, sans rien de personnel."
  columns:
    - name: cle_membre
      description: "La clé de substitution, md5 du numéro de la carte."
      data_tests:
        - unique
        - not_null
# 3. le fait : chaque clé recalculée existe dans sa dimension
- name: fct_ventes
  description: "Le fait des ventes, une ligne par ligne d'un ticket payé."
  columns:
    - name: cle_date
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_date')
              field: date_jour
    - name: cle_produit
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_produit')
              field: cle_produit
    - name: cle_membre
      description: "Vide pour un ticket sans carte de membre."
      data_tests:
        - relationships:
            arguments:
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
              to: ref('dim_membre')
              field: cle_membre
  # 4. le fait d'accumulation : une ligne par lot, et chaque clé dans sa dimension
  - name: fct_lots
    description: "Le fait d'accumulation des lots du torréfacteur, une ligne par lot, un jalon par colonne."
    columns:
      - name: lot_id
        description: "Le numéro du lot : le grain du fait."
        data_tests:
          - unique
          - not_null
      - name: cle_date
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_date')
                field: date_jour
      - name: cle_produit
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_produit')
                field: cle_produit
  # 5. le fait sans mesure : chaque clé dans sa dimension, et le statut dans une liste fermée
  - name: fct_consentements
    description: "Le fait sans mesure des consentements, une ligne par décision d'un membre."
    columns:
      - name: cle_date
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_date')
                field: date_jour
      - name: cle_membre
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_membre')
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
              field: cle_membre
      - name: statut
        description: "Ce que dit la décision : accordé, refusé ou retiré."
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: ['ACCORDE', 'REFUSE', 'RETIRE']
```

02_ae/tests/assert_temoin_consentements.sql

```sql
-- 02_ae/tests/assert_temoin_consentements.sql : la dernière décision de chaque membre, telle que sa fiche la porte
with journal as (
    -- 1. de chaque membre, la dernière décision du fait : accordée ou non
    select cle_membre, statut = 'ACCORDE' as consent
    from {{ ref('fct_consentements') }}
    qualify row_number() over (partition by cle_membre order by horodatage desc) = 1
),
fiche as (
    -- 2. le consentement que porte la fiche de chaque membre, sa clé recalculée
    select md5(membre_id) as cle_membre, consent_infolettre as consent
    from {{ ref('stg_membres') }}
)
-- 3. un membre d'un seul côté, ou deux réponses qui diffèrent : chaque ligne rendue est un échec
select coalesce(j.cle_membre, f.cle_membre) as cle_membre, j.consent as journal, f.consent as fiche
from journal as j
full outer join fiche as f on f.cle_membre = j.cle_membre
where j.cle_membre is null or f.cle_membre is null or j.consent <> f.consent
```


© 2026 iAlexMG. Tous droits réservés.